강화학습 Day2 보고서

1. MDP 구성 및 문제 정의

요소	정의
상태 S	FrozenLake 4x4 격자(0~15)
행동 A	0 = 왼쪽, 1 = 아래, 2 = 오른쪽, 3 = 위
전이 P	현재 상태 + 행동에 따라 Gym 환경이 반환하는 다음 상태
보상 R	일반 이동 = -1, 목표 상태(15) 도달 = +10, 구멍 = -10
할인율 gamma	0.95
정책(Policy)	학습 중: 엡실론-Greedy / 테스트 중: 엡실론 0으로 설정(무작위 값 방지)

2. Return과 Q-Learning

Return: 미래 보상의 할인 합
할인 Return: 미래 보상에 할인율 적용

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots$$

Q-learning: 각 상태 (s)에서 행동 (a)를 했을 때 앞으로 얻을 보상의 가치를 Q(s,a)로 추정하는 가치 기반 학습 방법
최적 행동 가치 q*(s,a): 현재 행동의 보상과 다음 상태에서 얻을 최선의 가치로 정의됨

3. Q업데이트 수치 예제 설명

[공식]

$$\text{target} = r + \gamma \max Q(s',a') \quad \dots(1)$$

$$\text{TD error} = \text{target} - Q(s,a) \quad \dots(2)$$

$$Q(s,a) \leftarrow Q(s,a) + \alpha \times \text{TD error} \quad \dots(3)$$

[가정]

- 현재 Q(s,a) = 0.2
- 즉시 보상 r = -1
- 다음 상태에서 가장 높은 Q값 max Q(s', a') = 0.8
- 학습률 α = 0.1
- 할인율 γ=0.95

1) 공식 (1) 적용: target = -1 + 0.95 × 0.80 = -0.24

```
next_q = np.max(self.q_table[:, n_row, n_col])
```

```
target = reward + self.gamma * next_q
```

2) 공식 (2) 적용: TD error = -0.24 - 0.2 = -0.44

3) 공식 (3) 적용: 새 Q(s,a) = 0.20 + 0.1 × (-0.44) = 0.156

```
self.q_table[action, row, col] = old_q + self.alpha * (target - old_q)
```

따라서 이 경험을 반영한 Q 값은 0.2에서 0.156으로 갱신됨.(예시)

4. **코드 구현**

1) 상태 및 행동 표현

```
row, col = divmod(state, 4)
```

행동은 총 4가지 이며 Q-table의 배열 모양은 (4, 4, 4)로 각각 행동, 행, 열

```
q_table[action, row, col]
```

따라서, 해당 칸에서 해당 행동을 선택했을 때의 가치 추정값으로 해석할 수 있음

2) Policy와 행동 선택

엡실론-Greedy 정책 사용함.

- 확률 ε로 네 행동 중 하나를 무작위로 선택해 탐험
- 확률 1 - ε로 현재 상태의 Q값이 가장 큰 행동을 선택해 활용

활용 단계에서 np.argmax가 가장 큰 Q값의 인덱스를 반환

Q값이 동점이면 가장 먼저 등장하는 인덱스를 선택

3) Q-learning 갱신

직전에 경험한 전이 하나도 Q-table을 갱신. 공식을 아래와 같음

$$q^*(s,a) = E[r + \gamma \max q^*(s',a')]$$

코드 내에서 구현

```
#Q-table 갱신
target = reward + self.gamma * next_q
self.q_table[action, row, col] = old_q + self.alpha * (target - old_q)
```

4) 학습 환경과 보상

```
reward = -1 # 움직일 때마다 -1
if(next_state == 15):   # 15번 위치에 도달하면 리워드 10
    reward = 10
    episode_success = True
elif(next_state == 5 or next_state == 7 or next_state == 11 or next_state == 12):   # 구멍에 빠지면 리워드-10
    reward = -10
```

- 움직일 때마다 -1 (한 걸음마다 작은 비용 부과 -> 에이전트가 가능한 목표에 빨리 도달하도록 하기 위함)
- 목표 도달: +10
- 구멍: -10

5) 학습 기록 및 결과 시각화

각 에피소트의 총 보상: episode_rewards
목표 도달 여부 = episode_succusses
이를 이용해 최근 200개 에피소드의 평균 보상과 성공률 계산

```
#학습 보상 및 성공률 기록
episode_rewards.append(episode_reward)
episode_successes.append(int(episode_success))
```

5. 실험 절차

1) 0으로 초기화된 Q-table로 에이전트 만듦
2) is_slippery = False 환경에서 5000 에피소드 학습
3) 매 에피소드 총 보상과 목표 도달 여부 기록
4) Q-table을 q_table.npy 로 저장
5) 각 행동 별 Q-value heatmap 및 200-episode rolling 평균 보상, 성공률 그래프 저장
6) 저장한 Q-table을 다시 읽고 엡실론 = 0인 greedy 정책으로 200회 평가
7) 목표 상태 도달 횟수와 성공률 출력

6. 실험 결과

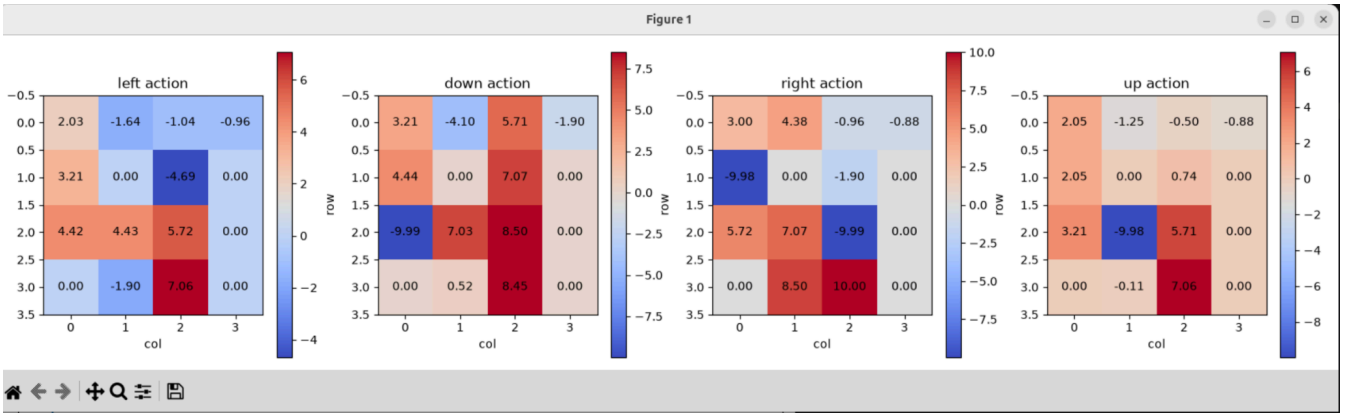
1) is_slippery = False인 경우

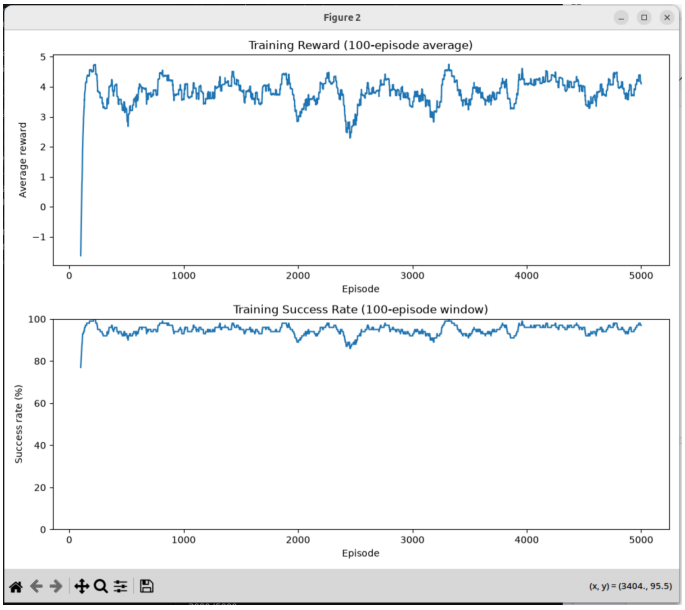
[기본 세팅]

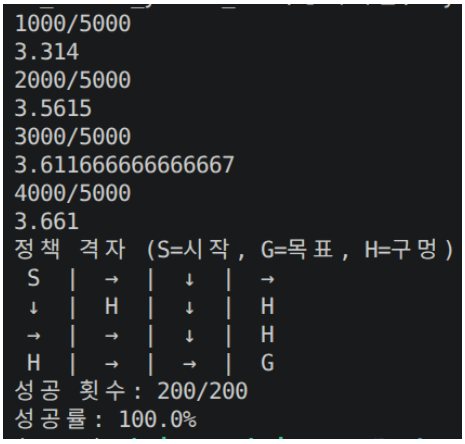
설정	값

환경	FrozenLake 4x4
미끄러짐	False
학습 에피소드	5000개
학습률	0.1
할인률	0.95
평가	200회

[결과]





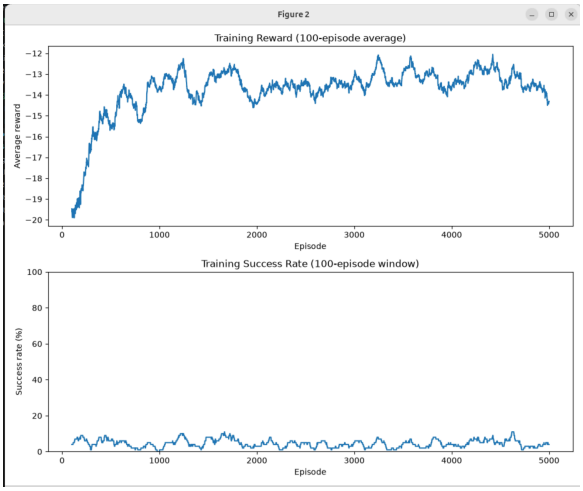


2) is slippery = True인 경우(다른 값 수정X)

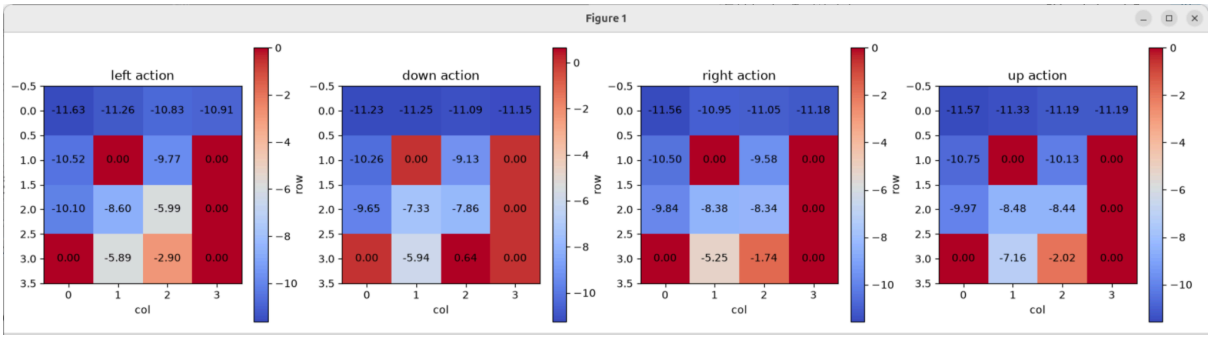
[기본 세팅]

설정	값
환경	FrozenLake 4x4
미끄러짐	**True**
학습 에피소드	5000개
학습률	0.1
할인률	0.95
평가	200회

[결과]




```
1000/5000
-15.551
2000/5000
-14.493
3000/5000
-14.185
4000/5000
-13.94375
정책 격자 (S=시작, G=목표, H=구멍)
 S  |  →  |  ←  |  ←
 ↓  |  H  |  ↓  |  H
 ↓  |  ↓  |  ←  |  H
 H  |  →  |  ↓  |  G
성공 횟수: 19/200
성공률: 9.5%
```



[결론]

성공할 확률이 거의 없고 정책 격자도 제대로 생성이 안됨.
정책 격자가 제대로 생성 안됐는데 성공한 횟수가 19회 인것으로 보아 운으로 성공했다고 판단할 수 있음.
평균 보상은 False인 경우와 큰 차이점은 없어보였지만 성공률이 현저히 낮은 것을 확인할 수 있었다.

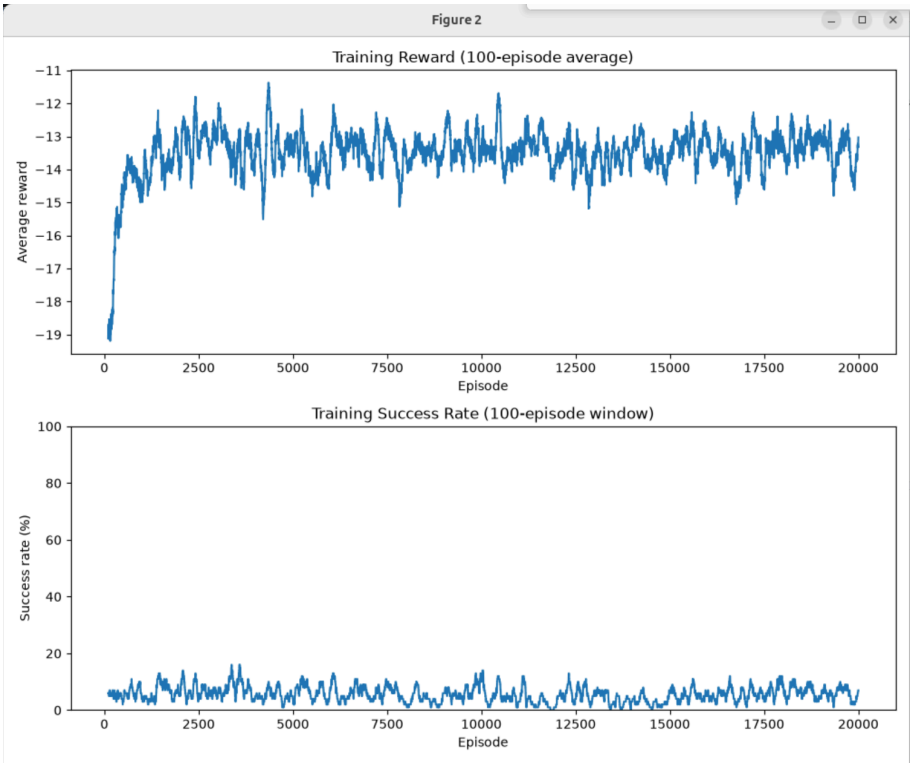
Q. is_slippery만 True로 바꾸고 학습 돌리니 제대로 학습이 안됨. 어떤 값을 바꿔야 미끄러짐 환경에서도 학습이 될까

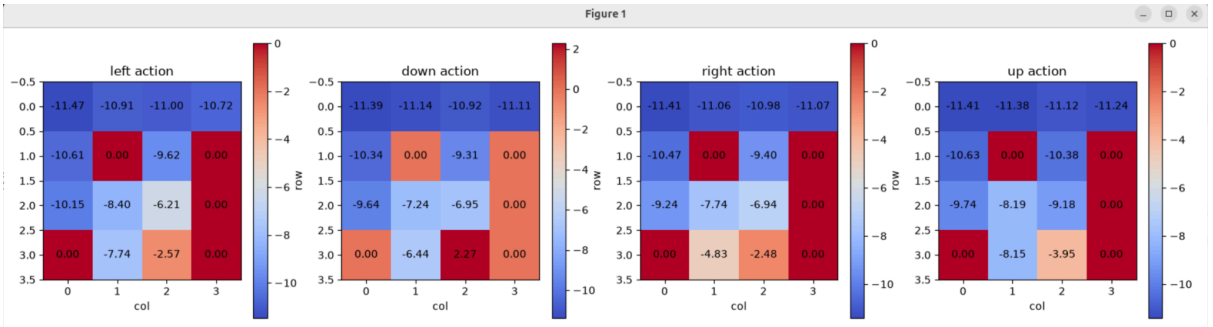
1. 학습 에피소드 수 늘려보기(5,000 -> 20,000 episode)

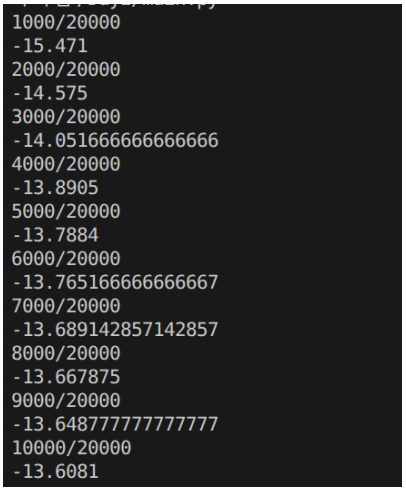
설정	값
환경	FrozenLake 4x4
미끄러짐	True
학습 에피소드	20,000개
학습률	0.1
할인률	0.95

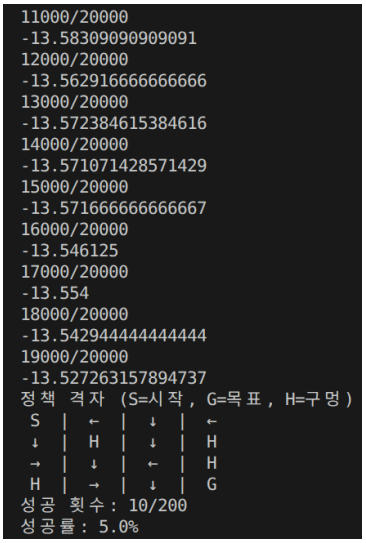
엡실론	0.9(에피소드마다 0.05씩 감소 + 곱셈으로 줄이기)
평가	200회

[결과]









[결론]

정책 격자가 조금 더 정확한 방향으로 만들어진 것을 확인할 수 있었음

그러나, 실제 성공 횟수는 True로만 바꿨을 때보단 더 적었음

개인적으로 생각하는 원인은 엡실론 값을 에피소드마다 0.05씩 감소시키기 때문에 약 17~18회만에 엡실론 값이 0.05 이하로 떨어집니다. 따라서 나머지 19,980개의 에피소드에서는 탐험률이 5%에 머무르기 때문에 엡실론 값을 떨어뜨리는 방식을 바꿔야겠다고 생각했습니다.

2. 엡실론 감소 방식 바꿔보기

설정	값	변경
환경	FrozenLake 4x4	동일
미끄러짐	True	동일
학습 에피소드	20,000개	동일
학습률	0.1	동일
할인률	0.95	동일
엡실론	0.9(에피소드마다 0.05씩 감소 + 곱셈으로 줄이기)	곱셈으로 줄이기만 사용
평가	200회	동일

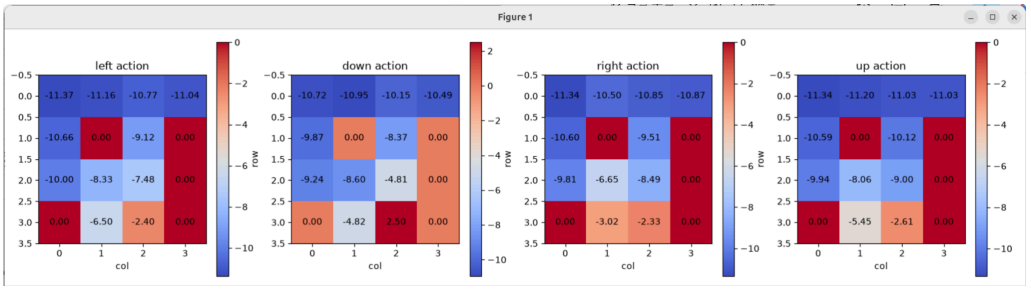
**곱셈을 줄인다는 의미:

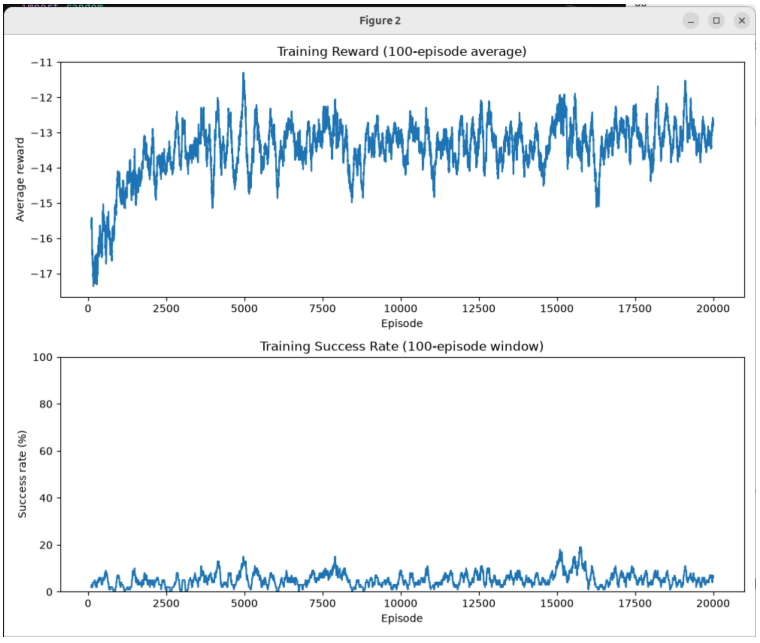
```
self.eps -= 0.05
self.eps = max(0.05, self.eps * 0.999)
```

⇒

```
self.eps = max(0.05, self.eps * 0.999)
```

[결과]






```
1000/20000
-15.822
2000/20000
-15.0215
3000/20000
-14.572
4000/20000
-14.302
5000/20000
-14.0642
6000/20000
-13.959666666666667
7000/20000
-13.872857142857143
8000/20000
-13.765
9000/20000
-13.755333333333333
10000/20000
-13.6996
```

```
10000/20000
-13.6996
11000/20000
-13.666454545454545
12000/20000
-13.647166666666667
13000/20000
-13.608923076923077
14000/20000
-13.595642857142858
15000/20000
-13.581933333333334
16000/20000
-13.5459375
17000/20000
-13.537823529411765
18000/20000
-13.517777777777777
19000/20000
-13.49178947368421
정책 격자 (S=시작, G=목표, H=구멍)
 S  |  →  |  ↓  |  ↓
 ↓  |  H  |  ↓  |  H
 ↓  |  →  |  ↓  |  H
 H  |  →  |  ↓  |  G
성공 횟수: 7/200
성공률: 3.5%
```

[결론]

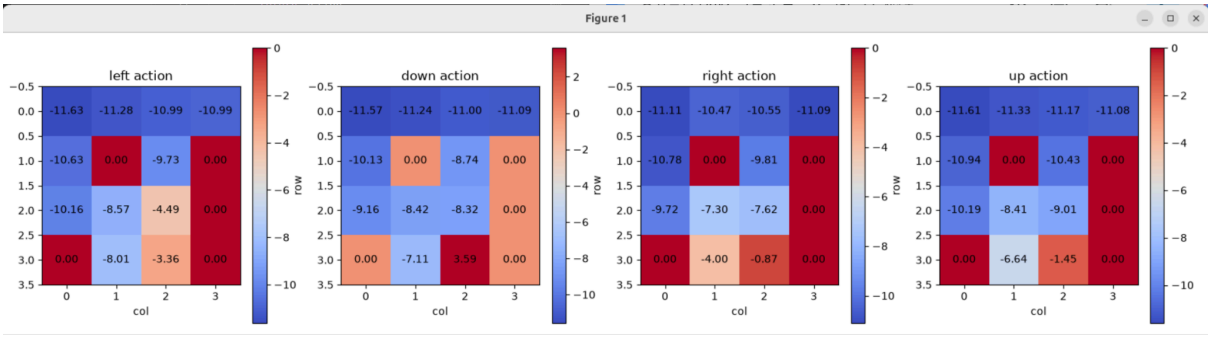
정책 격자 방향을 더 정확히 학습된거같지만 성공률은 오히려 줄어듦.

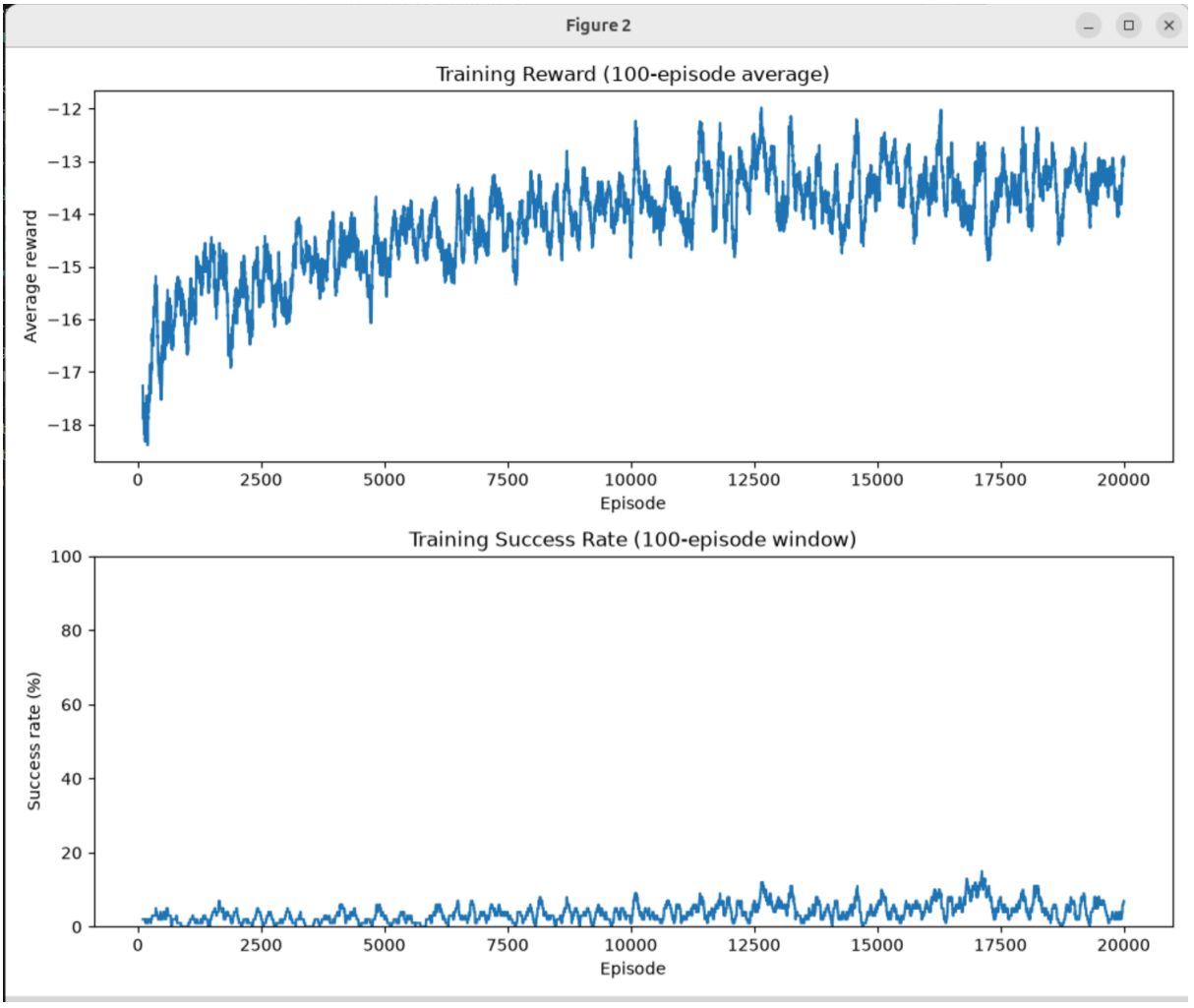
AI에게 0.999였던 감소 배수를 0.99985로 수정하라는 조언을 받아 다시 실험 시도

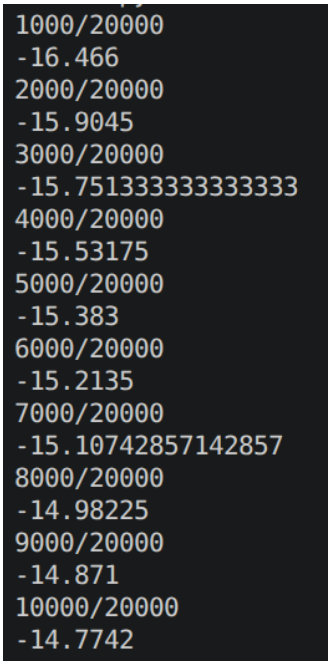
3. 감소 배수 0.999 ⇒ 0.99985

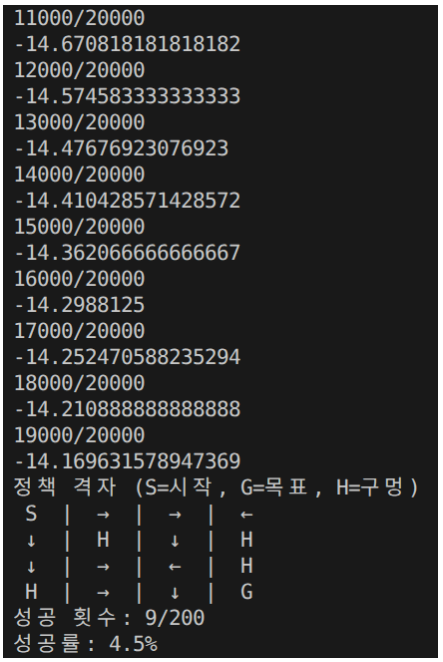
설정	값	변경
환경	FrozenLake 4x4	동일
미끄러짐	True	동일
학습 에피소드	20,000개	동일
학습률	0.1	동일
할인률	0.95	동일
엡실론	0.9(에피소드마다 0.05씩 감소 + 곱셈으로 줄이기)	곱셈으로 줄이기만 사용
감소 배수	0.999	0.99985
평가	200회	동일

[결과]


Figure 1
left action
down action
right action
up action
row
col


Figure 2
Training Reward (100-episode average)
Average reward
Episode
Training Success Rate (100-episode window)
Success rate (%)
Episode



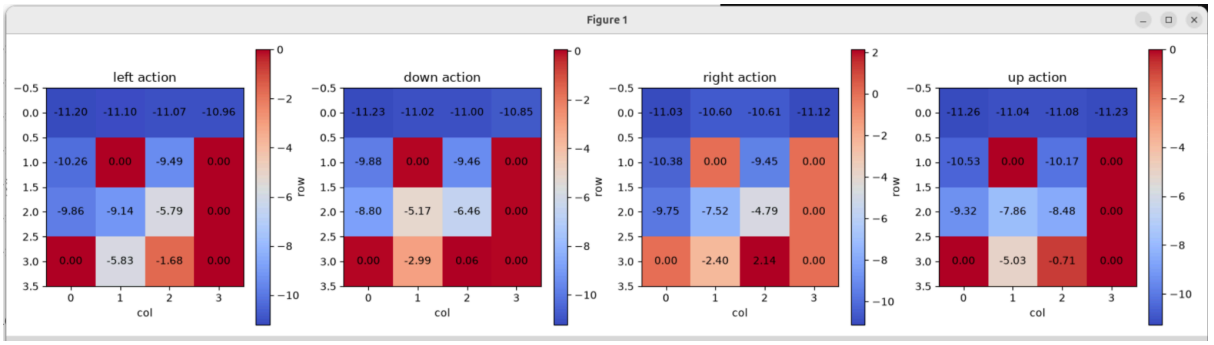


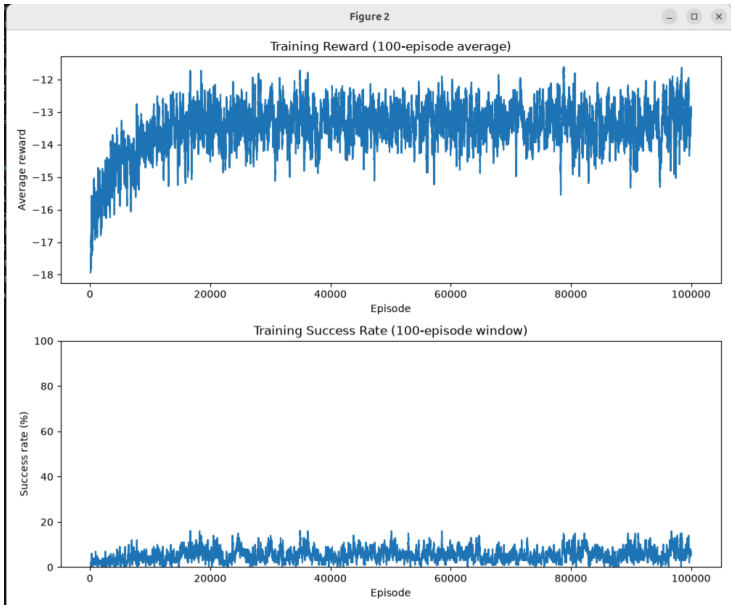
[결론]
큰 변화 없음.
학습 에피소드 수를 20,000보다 훨씬 더 높이면 어떻게 될지 다시 테스트 해보기로함

4. 에피소드 수 훨씬 더 증가(20,000 ⇒ 100,000개)

설정	값	변경
환경	FrozenLake 4x4	동일
미끄러짐	True	동일
학습 에피소드	20,000개	100,000개
학습률	0.1	동일
할인률	0.95	동일
엡실론	0.9(에피소드마다 0.05씩 감소 + 곱셈으로 줄이기)	곱셈으로 줄이기만 사용
감소 배수	0.999	0.99985
평가	200회	동일

[결론]






```
정책 격자 (S=시작, G=목표, H=구멍)
 S  |  →  |  →  |  ↓
 ↓  |  H  |  →  |  H
 ↓  |  ↓  |  →  |  H
 H  |  →  |  →  |  G
성공 횟수: 6/200
성공률: 3.0%
```

[결론]

학습 에피소드를 100,000개로 늘려도 성공률이 증가하지 않음

기존 보상 체계에 문제가 있다고 생각이 듦
왜냐면 미끄러짐이 없는 경우에는 일반 스텝을 -1로 설정해서 너무 많은 길을
우회하지않도록 코딩했는데 미끄러짐이 있는 경우에는 계속 돌고 돌아 도착하기만 하면 됨.
따라서 일반 스텝의 보상을 0으로 설정하고 목표에 도달하면 보상 1을 주는 형식으로
수정해보기로함

5. 보상 체계 수정

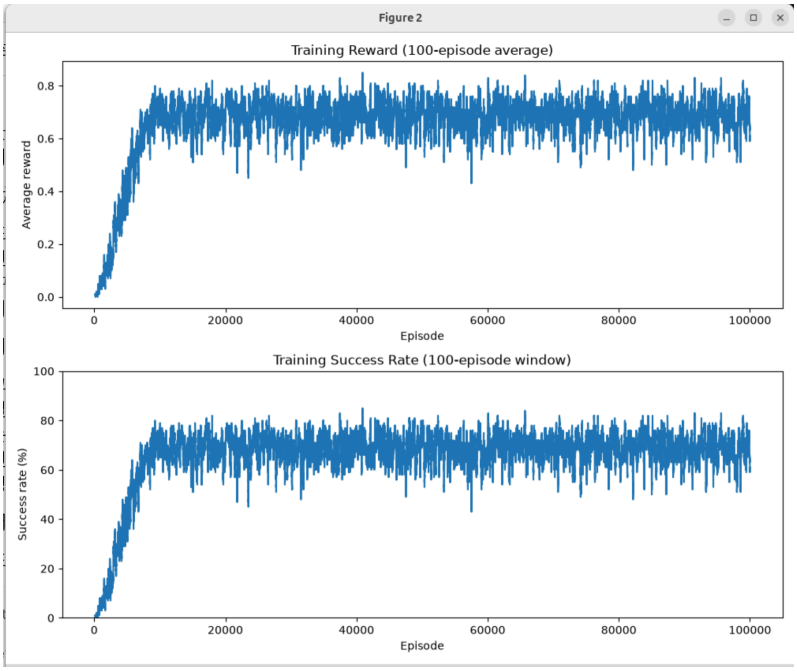
일반 스텝 = 0, 구멍에 빠졌을 때도 0

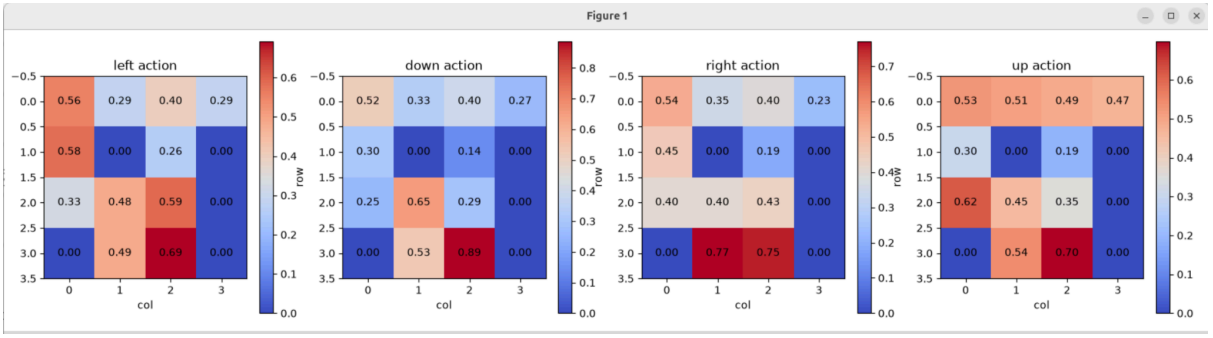
**왜 구멍에 빠져도 0인가

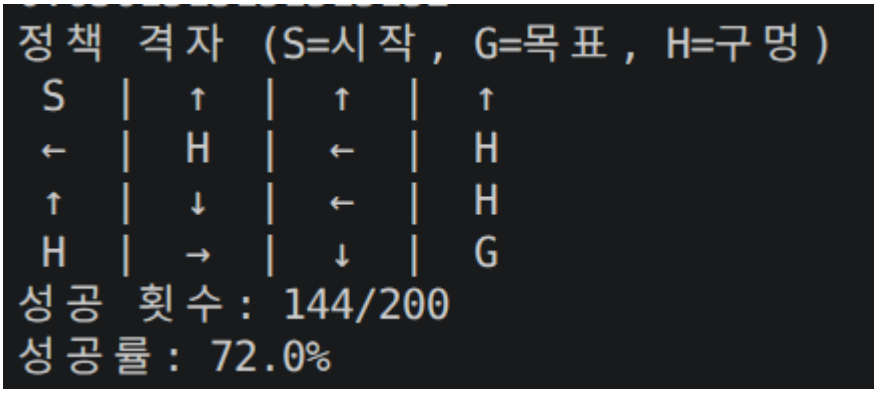
마이너스를 주게되면

설정	값	변경
환경	FrozenLake 4x4	동일
미끄러짐	True	동일
학습 에피소드	20,000개	**100,000개**
학습률	0.1	동일
할인률	0.95	**0.99**
엡실론	0.9(에피소드마다 0.05씩 감소 + 곱셈으로 줄이기)	**곱셈으로 줄이기만 사용**
감소 배수	0.999	**0.9995**
보상 체계	일반 스텝 = -1	**0으로 수정**
평가	200회	동일

[결과]







[결론]

성공률이 72%까지 증가하였습니다.

이전 보상 체계에서 어떤 점이 문제였는지 생각해보면

만약 매 스텝마다 -1을 받는다면 오래 헤맬수록 값이 계속 줄어듦
만약 계속 헤매다가 -20이 되어버리면 -10인 구멍보다 값이 더 작기 때문에
헤매는것보다 구멍에 빠지는 것을 선택했을 가능성이 크다고 생각됩니다.
따라서 미끄러짐 환경에서는 목표에 도달하는 것이 목표이지 빨리 도달하지 않아도
되기때문에 일반 스텝을 0으로 설정하는 것이 매우 중요했던 것 같습니다.

7. AI 사용

- 사용 목적: 미끄러짐이 존재하는 환경에서 학습시킬 때 어떤 값을 수정하는게 좋을지 조언 구하기 위해

- 학습 에피소드 수, 엡실론 감소 방식을 수정해서 테스트 해봐도 성공률이 오르지 않아 어느 값을 바꾸는게 좋을지 조언 받음

시도1. 학습 에피소드 수 20,000개로 증가
시도2. 엡실론 값 곱셈으로만 감소시킴(-0.05 제거)

이렇게 is_slippery = True 환경에서 실행시켰을 때 제대로 학습시킬 수 있도록 시도하고있음
위에 두가지 시도해봤는데 성공률이 오르지않음

어느 값을 바꿔야할지 조언해줘

5:07 PM

Explaining epsilon schedule and rewards

정책 격자가 더 좋아 보이는데도 성공률이 낮을 수 있습니다. `is_slippery=True` **에서는 같은 방향을 선택해도 미끄러질 수 있으므로, 화살표가 목표를 향한다는 것만으로 성공이 보장되지는 않습니다.** 또 평가 성공률은 환경의 무작위성 때문에 실행마다 변동합니다.

현재 코드를 확인하면 학습은 20,000회지만, ε는 `0.9 × 0.999^n` 으로 감소하다가 약 **2,900회 후 0.05에 도달**합니다. 나머지 대부분의 에피소드에서는 탐험률이 5%에 머뭅니다. 따라서 다음 실험은 여러 값을 한꺼번에 바꾸지 말고, 아래 순서로 **한 조건씩** 바꿔 비교하는 걸 권합니다.

다음 실험: ε 감소를 더 천천히

- 시작 ε: 우선 `0.9` 유지
- 하한 ε: `0.05` 유지
- 감소 배수: 현재 `0.999` → `0.99985`
- 학습 횟수: `20,000` 유지
- α= `0.1` , γ= `0.95` 는 그대로

현재 `decrease_eps()` 가 있는 39–41행에서 이 실험값을 적용하면 ε가 약 19,000회쯤에 0.05에 도달하므로, 학습 기간의 더 많은 부분에서 탐험이 유지됩니다. 기존 `0.05` 를 빼는 줄은 이미 제거된 상태여야 합니다.

그래도 성공률이 낮으면

1. **학습 성공률 곡선이 마지막에도 오르고 있다면** 학습 에피소드 수를 `20,000 → 50,000` 으로 늘려봅니다. 성공률이 이미 평평하다면 에피소드만 늘려도 효과가 작을 수 있습니다.
2. **성공률이 들쭉날쭉하면** 별도 실험에서 α를 `0.1 → 0.05` 로 낮춰 비교합니다. Q값 변화가 완만해질 수 있지만 수렴은 더 느릴 수 있습니다.
3. **미래 목표를 더 중시하는 설정을 시험하고 싶다면** γ를 `0.95 → 0.99` 로 비교할 수 있습니다. 다만 γ를 바꾸면 학습 특성도 달라지므로 α와 동시에 바꾸지는 마세요.
4. 평가가 200회라면 성공률 추정에 변동이 있을 수 있습니다. 과제의 200회 결과는 유지하고, 추가 분석에서는 같은 설정을 여러 번 학습하거나 평가 횟수를 늘려 평균도 확인하세요.

우선 추천하는 비교는 **현재 설정과** `ε 감소 배수만 0.99985로 바꾼 설정` **입니다. 마지막 100회 학습 성공률과 greedy 평가 성공률을 함께 비교하세요. 보상 설계를 바꾸는 것은 에이전트가 최적화하는 목표 자체를 바꾸므로, 이 비교를 먼저 마친 뒤 별도 실험으로 다루는 편이 좋습니다.**